5010 QDD Actuator V2: Build and Operating Guide

A 3D-printed quasi-direct-drive robot joint built on a 5010 gimbal motor, a SimpleFOC driver and a 6:1 planetary gearbox. Last updated: November 8, 2026.

V2 clamps the big output bearing on all four faces, so pushing, pulling and tipping loads on the output go into the housing instead of the motor. The V1 design is kept in the CAD/V1 folder, and its guide is the document "5010".

What to print for V2

Part	From V1	File (CAD/V2)
Carrier	Reprint: hub cut down, six new screw holes	PlanetaryGearBox 5010-Carrier.3mf
Ring gear	Reprint: top lip removed, new shoulder under the bearing, six new screw holes	PlanetaryGearBox 5010-Ring.3mf
Output flange	New	PlanetaryGearBox 5010-Output Flange.3mf
Bearing retainer	New	PlanetaryGearBox 5010-Bearing Retainer.3mf
Arm A1	New, optional (section 4, step 8)	PlanetaryGearBox 5010-Arm A1.3mf
Scale pusher	New, optional, for torque calibration	PlanetaryGearBox 5010-Arm A1 Pusher.stl

First calibration arm	Superseded by Arm A1, still fits	PlanetaryGearBox 5010-Arm.3mf
Base, sun (+2 mm), planets, rotor adapter	Reuse the V1 prints	Same files as V1

All bearings are reused. New screws: 6 × M3 × 12 (flange) and 6 × M3 × 10 (retainer), plus 8 × M3 × 8 for Arm A1 and the scale pusher.

1. Overview

This guide explains how to build a compact robot joint actuator from a 5010-size brushless motor. A low-ratio planetary gearbox is bolted directly onto the motor. This layout is called a quasi-direct-drive (QDD) actuator, the same idea used in legged robots such as the MIT Mini Cheetah.

Because the gear ratio is low (6:1), the joint has three useful properties:

- **Back-drivable:** you can turn the output by hand, so the joint is safe and compliant.
- **Torque without a torque sensor:** motor current maps closely to output torque, so the joint can "feel" loads.
- **Fast and responsive:** there is little friction or reflected inertia.

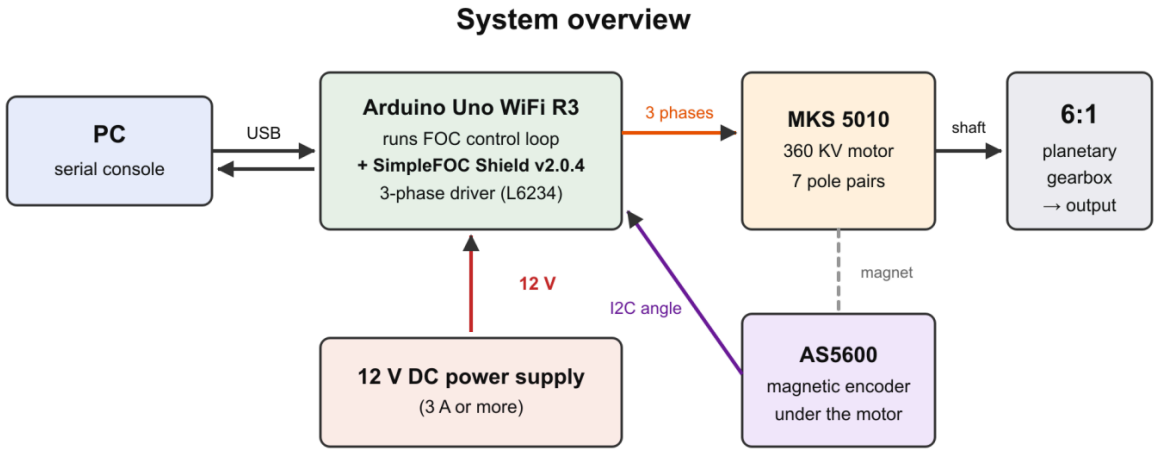


Figure 1. How the parts connect.

An Arduino Mega reads the motor angle from a magnetic encoder and the motor currents from the shield's current sensors, about 380 times per second. It runs field-oriented control (FOC), which applies smooth, precisely timed voltages to the three motor phases. On top of that it runs angle, speed or torque control, and it takes commands from a PC over USB.

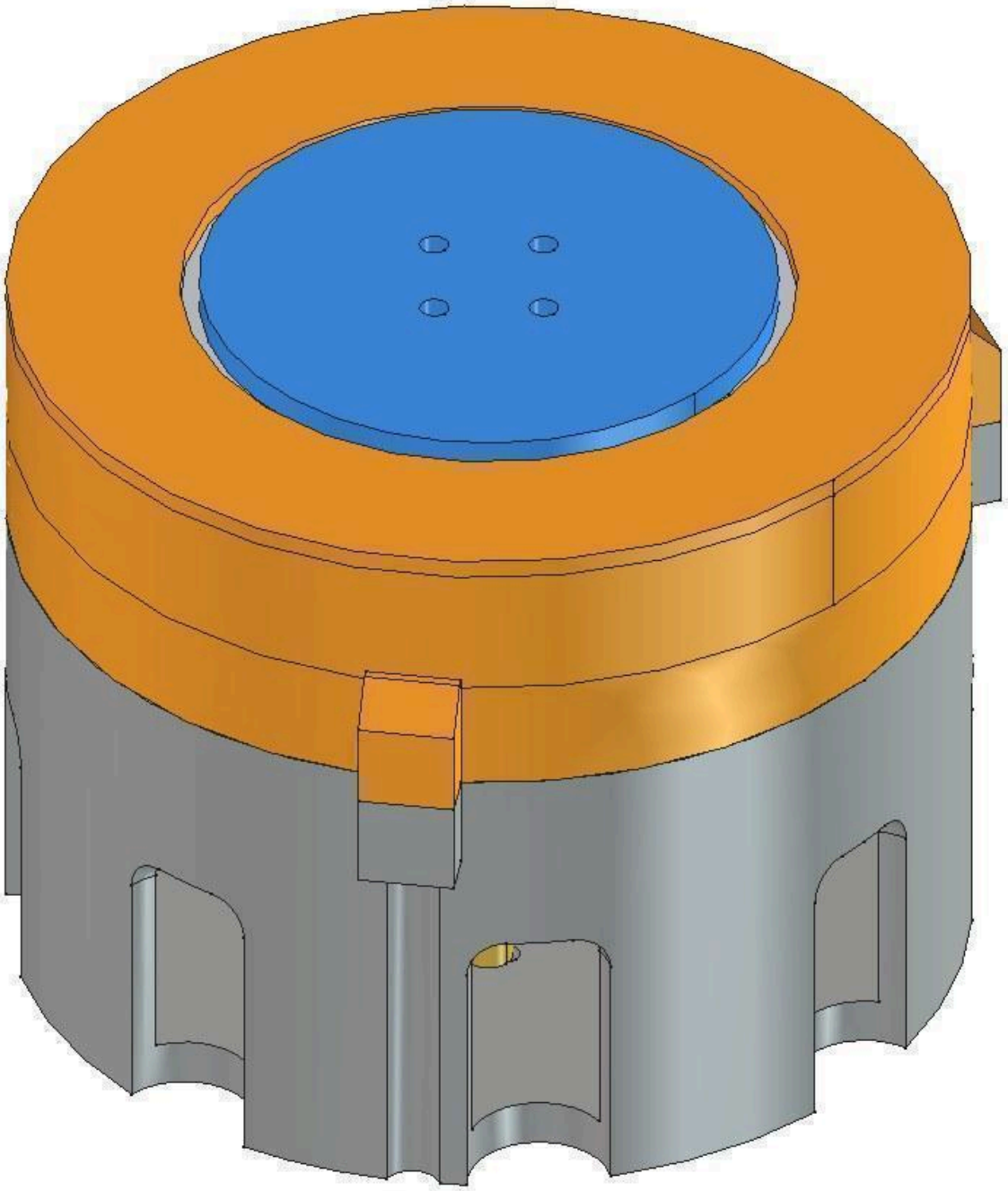


Figure 2. The finished actuator: printed base (gray), ring gear (orange) and carrier output (blue). Figures 2–12 show the V1 parts; Figure 5b shows the V2 output bearing.

Specifications

Gear ratio	6 : 1, single-stage planetary, helical teeth

Size	About 105 mm across the tabs × 89 mm tall (V2 adds 5 mm for the output flange)
Supply	12 V DC (the motor is rated 12 to 24 V)
Output torque	About 0.26 Nm measured at the default 3.5 A current cap (0.099 Nm per amp above 0.87 A of gearbox friction), about 0.31 Nm at the 4 A maximum (C4). The SimpleFOC Shield's driver is the limit, not the motor or the gearbox. See section 12.
Output speed	Angle moves are capped at 20 rad/s at the motor (about 32 RPM at the output) by the V setting. The S command spins the output at a set speed. The 3 V voltage cap also limits the top speed in velocity mode.
Position sensing	12-bit magnetic encoder on the motor: about 0.09° at the motor, about 0.015° at the output
Measured response	With the gearbox fitted, 60° output moves settle in about 0.3 s with no overshoot and under 0.2° final error. Holding still, the output moves less than 0.1°.
Interface	USB serial, 115200 baud, text commands

2. Parts and tools

2.1 Electronics

Qty	Part	Notes
1	MKS 5010 360 KV gimbal motor with MKS AS5600 encoder	Sold as a kit with the encoder board, brass legs and cables

1	SimpleFOC Shield v2.0.4	Three-phase driver (L6234) with inline current sensors, plugs onto the Arduino
1	Arduino Mega 2560	The firmware no longer fits an Uno (it uses about 47 KB of flash). The build started on an Uno WiFi R3 and moved to the Mega on October 30.
1	12 V DC power supply, 3 A or more	With bare-wire or screw-terminal leads
1	USB data cable (USB-B)	Charge-only cables will not work
4	Female-to-female jumper wires	Or use the kit's 6-pin PH2.0 cable

2.2 Bearings

Qty	Bearing	Size (ID × OD × width)	Where
3	608 (2RS or ZZ)	8 × 22 × 7 mm	One inside each planet gear
1	MR105 (ZZ)	5 × 10 × 4 mm	Center of the carrier, supports the sun gear's pin
1	61812 (6812-2RS)	60 × 78 × 10 mm	Output bearing between carrier and ring

2.3 Screws

Qty	Screw	Use

3	M4 × 10 button head + M4 washer (9 mm OD)	Clamp each planet bearing onto its carrier pin (do not use M4 × 12: it bottoms out)
4	M3 × 8 socket head	Rotor adapter to motor rotor. The adapter clamps 4.5 mm, so an M3 × 8 goes 3.5 mm into the rotor. The rotor's threads are only 5 mm deep: never use a screw longer than M3 × 9.
4	M3 × 6–8 socket head	Motor (encoder legs) to base floor
3	M3 × 16 socket head	Base tabs to ring tabs
6	M3 × 12 socket head	Output flange to carrier (V2). The heads sit in 3.5 mm counterbores.
6	M3 × 10 socket head	Bearing retainer to ring (V2)
4	M3 × 8 socket head	Arm A1 to output flange (V2). The heads sit in the arm's counterbores; the flange holes are 7 mm deep, so M3 × 10 is too long.
4	M3 × 8 socket head	Scale pusher to Arm A1, from the top through the arm's far counterbores into the pusher's 2.9 mm holes.

2.4 Printed parts

Qty	Part	File
1	Base	PlanetaryGearBox 5010-Base.3mf

1	Ring gear (V2: no top lip, bearing shoulder inside)	PlanetaryGearBox 5010-Ring.3mf
1	Carrier (V2: hub cut down)	PlanetaryGearBox 5010-Carrier.3mf
1	Output flange (V2, new)	PlanetaryGearBox 5010-Output Flange.3mf
1	Bearing retainer (V2, new)	PlanetaryGearBox 5010-Bearing Retainer.3mf
1	Arm A1 (new, optional)	PlanetaryGearBox 5010-Arm A1.3mf
1	Scale pusher (new, optional)	PlanetaryGearBox 5010-Arm A1 Pusher.stl
3	Planet gear	PlanetaryGearBox 5010-Planet 1/2/3.3mf (identical)
1	Sun gear (12.5 mm of teeth, 17 mm overall)	**PlanetaryGearBox 5010-Sun.3mf**. This is the sun that fits the assembled gearbox. It is the +2 mm version; the older, shorter suns were removed.
1	Rotor adapter	PlanetaryGearBox 5010-Rotor Adapter.3mf

The source CAD file is **PlanetaryGearBox 5010.FCStd** (FreeCAD 1.1) in the project's **CAD/V2** folder, together with all the print files. The V1 design is in CAD/V1.

2.5 Tools

- Hex keys: 2, 2.5 and 3 mm
- Small flat screwdriver, for the shield's screw terminals

- Calipers
- Vise or clamps, to hold the actuator during testing
- Fine sandpaper or a small file
- A socket that fits the bearing outer rings, for pressing bearings in

3. Printing

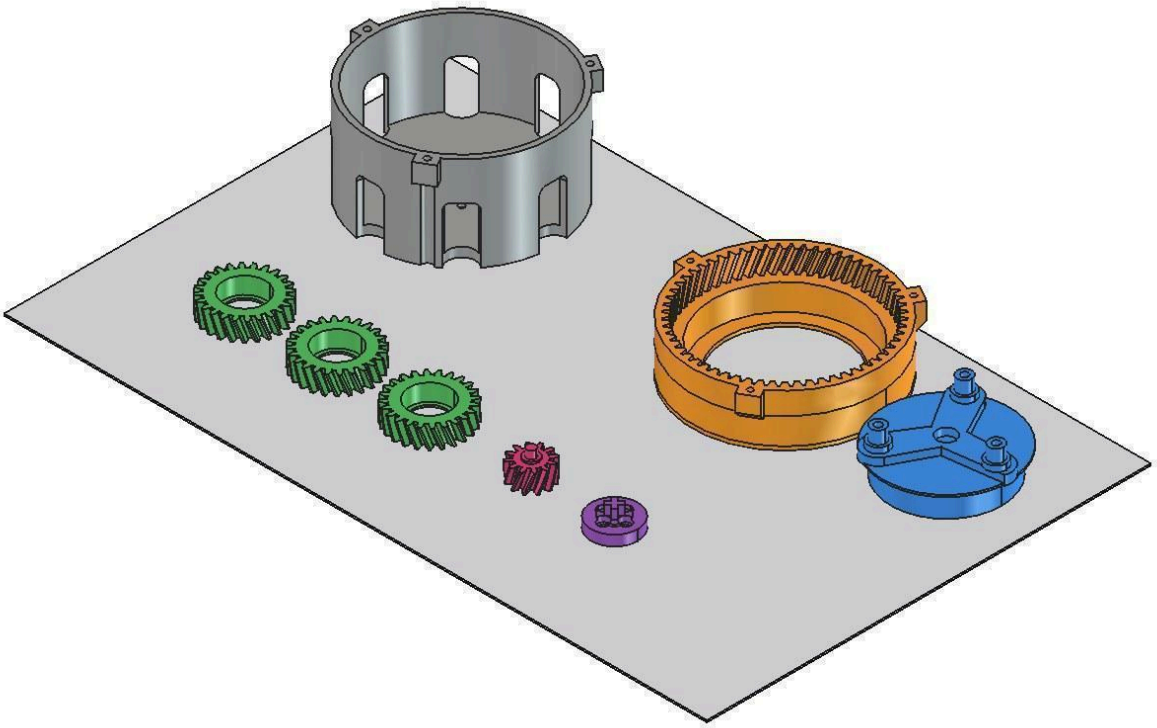


Figure 3. The parts in their print orientation. The 3MF files are already oriented this way.

Setting	Recommended
Material	PETG or PLA+ (plain PLA works for testing)
Layer height	0.16–0.2 mm; use 0.12–0.16 mm for the sun and planets
Walls / top / bottom	4 walls, 5 top and 5 bottom layers

Infill	40–50% gyroid for housings; 60–100% for the sun, planets and carrier
Supports	Build plate only. Needed under the bolt tabs on the base and under the carrier's outer rim. The V2 ring, flange and retainer need none.
Elephant-foot compensation	Turn it on. A flared first layer on gear teeth causes binding.

Orientation:

- The **ring** is printed gear end down, with the bolt tabs on the bed. The bearing shoulder inside has a 45° underside, so it needs no support.
- The **carrier** is printed with its flat output face on the bed and the pins pointing up.
- The **output flange** is printed with its arm face (the side with the counterbores) on the bed.
- The **bearing retainer** is a flat ring and prints either way up.
- The **sun** is printed with its key pocket on the bed. The pocket entrance has a 0.4 mm chamfer so elephant foot does not close it.
- All other parts print as modeled.

3.1 Test fits (do these first)

Print one planet, the sun and the ring first, then check each of the following:

1. **608 in the planet:** the bearing should press in firmly by hand or with a vise. If it falls in loosely, use a drop of retaining compound. If it will not go in, sand the bore lightly.
2. **Carrier pin in the 608:** the bearing should slide onto the 8 mm pin with almost no play.
3. **Gear mesh:** place the planet between the sun and the ring. It should turn smoothly with a little free play. The teeth are designed with 0.1 mm of backlash.
4. **Sun pin in the MR105:** the φ4.8 × 4 mm pin should slide into the bearing freely.
5. **61812 in the ring:** the bearing should drop into the ring from the top and sit flat on the shoulder, snug but without force.
6. **Carrier hub in the 61812:** the Ø60 hub should push through the bearing by hand, snug. If it is too tight, sand the hub lightly.
7. **Sun on the adapter key:** the 7.7 × 2.7 mm cross key (4 mm tall) should slide into the sun's 8 × 3 mm pocket without force.

4. Mechanical assembly

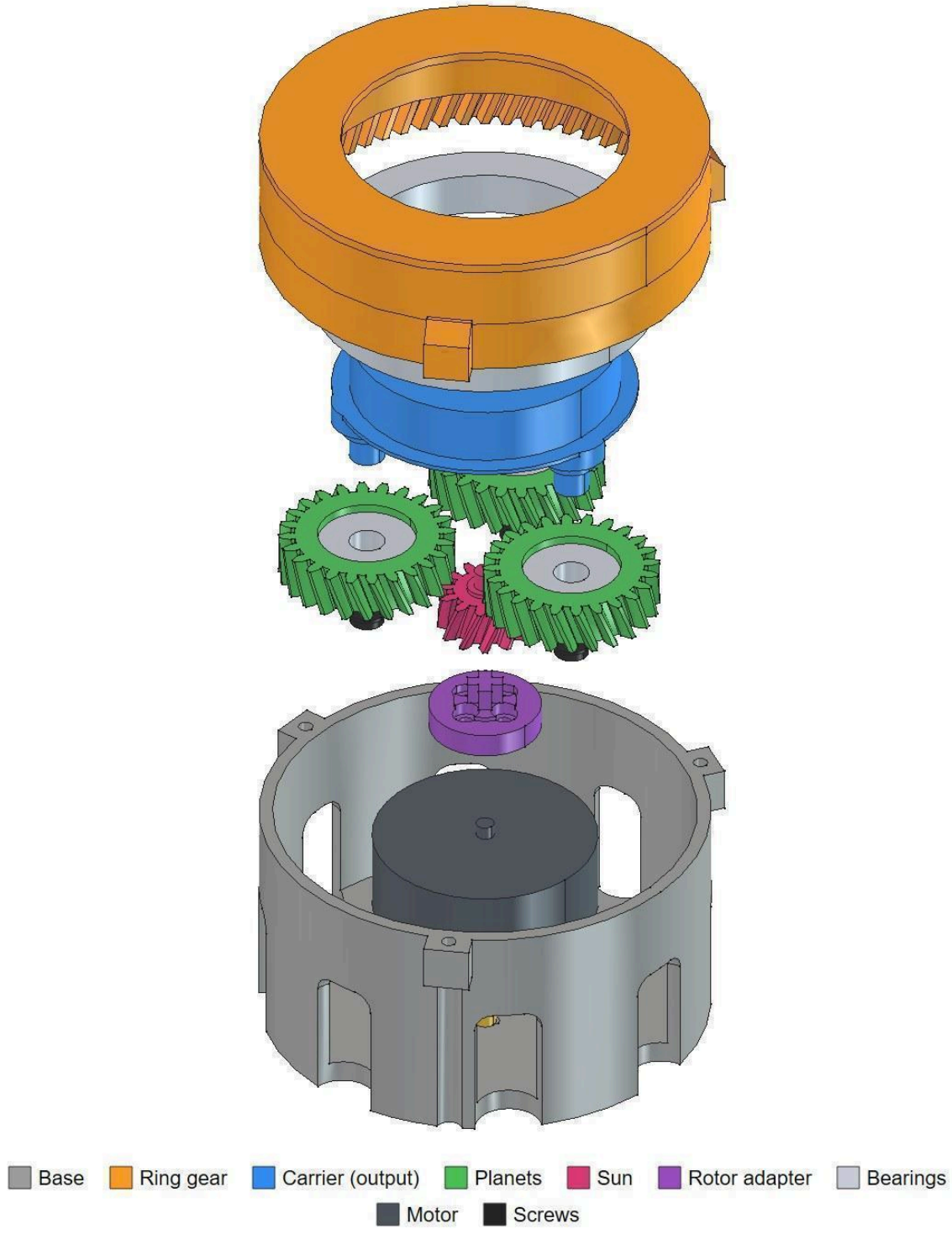


Figure 4. Exploded view, in assembly order from bottom to top.

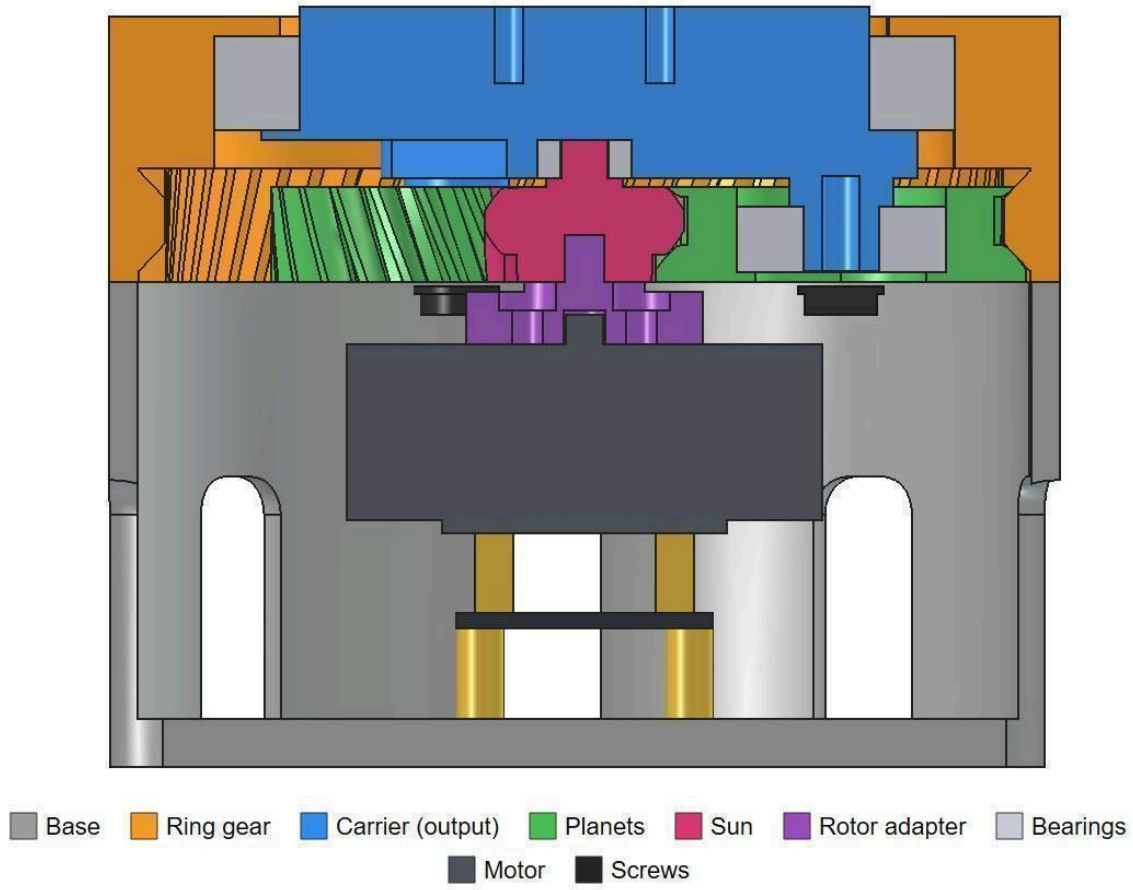


Figure 5. Cutaway through the assembled actuator (V1).

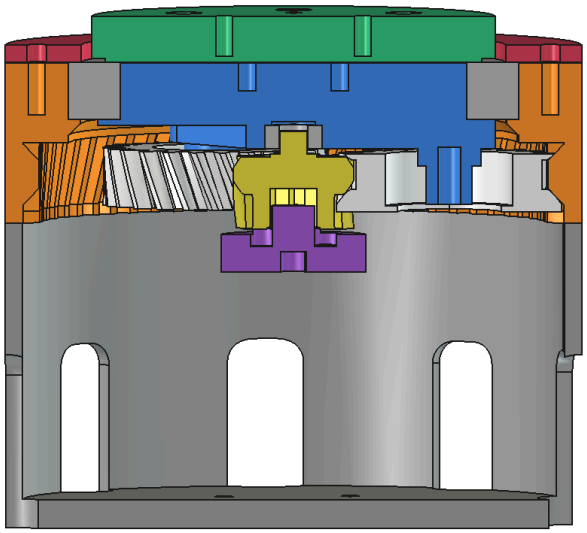


Figure 5b. V2 cutaway. The output bearing (gray) is clamped by the output flange (green) and the carrier (blue) on its inner ring, and by the retainer (red) and the ring's shoulder (orange) on its outer ring.

The motor stands on its encoder legs inside the base. The rotor adapter is screwed to the rotor and drives the sun through a cross-shaped key. The three planets mesh with the sun and with the fixed ring. They carry the carrier around, and the carrier is the output.

Axial stack, from the base top: the sun's teeth cover 9.5 of the planets' 10 mm face. A 0.5 mm spacer on top of the sun leaves 1 mm of clearance below the MR105, and the sun's pin reaches 3 mm into the bearing.

Output bearing (V2): the 61812 is clamped on all four faces. Its inner ring sits between the carrier's rim below and the output flange above; its outer ring sits between the ring's shoulder below and the retainer above. The carrier hub and the ring top stop 0.2 mm below the bearing top, so tightening the flange and retainer screws clamps the bearing rings.

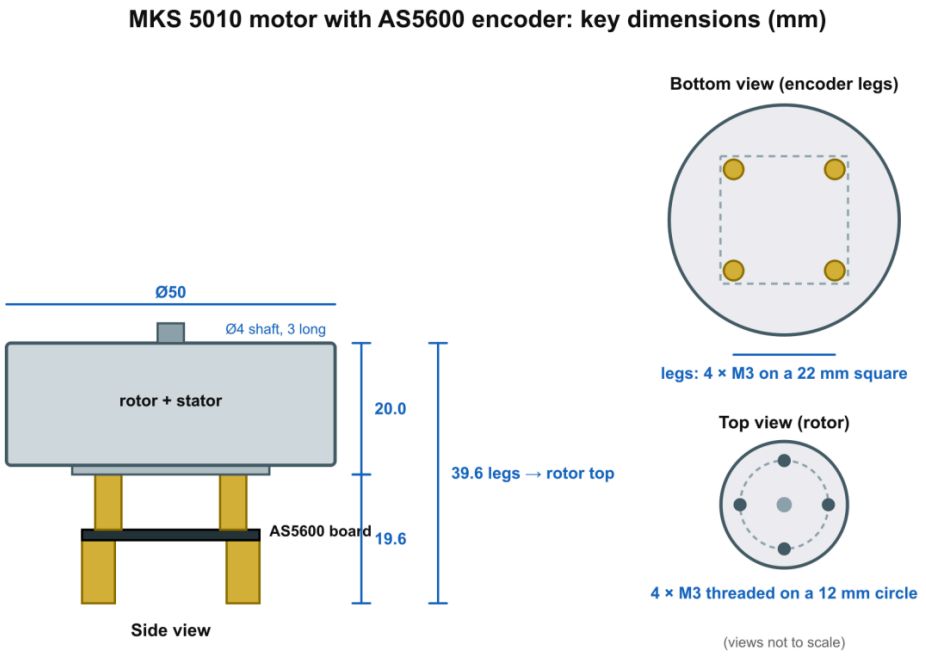


Figure 6. Motor and encoder dimensions used by the design.

Step 1. Mount the motor in the base

1. Before mounting, spin the motor by hand. It must turn freely with no scraping. A bell pressed too far down onto the stator rubs the windings, which makes the motor weak and rough.
2. Feed the motor's 3 phase wires and the encoder cable out through one of the windows in the base wall.
3. Stand the motor on its four brass legs over the four counterbored holes in the base floor (22 mm square).
4. From underneath, screw 4 × M3 screws through the floor into the legs. Tighten firmly.

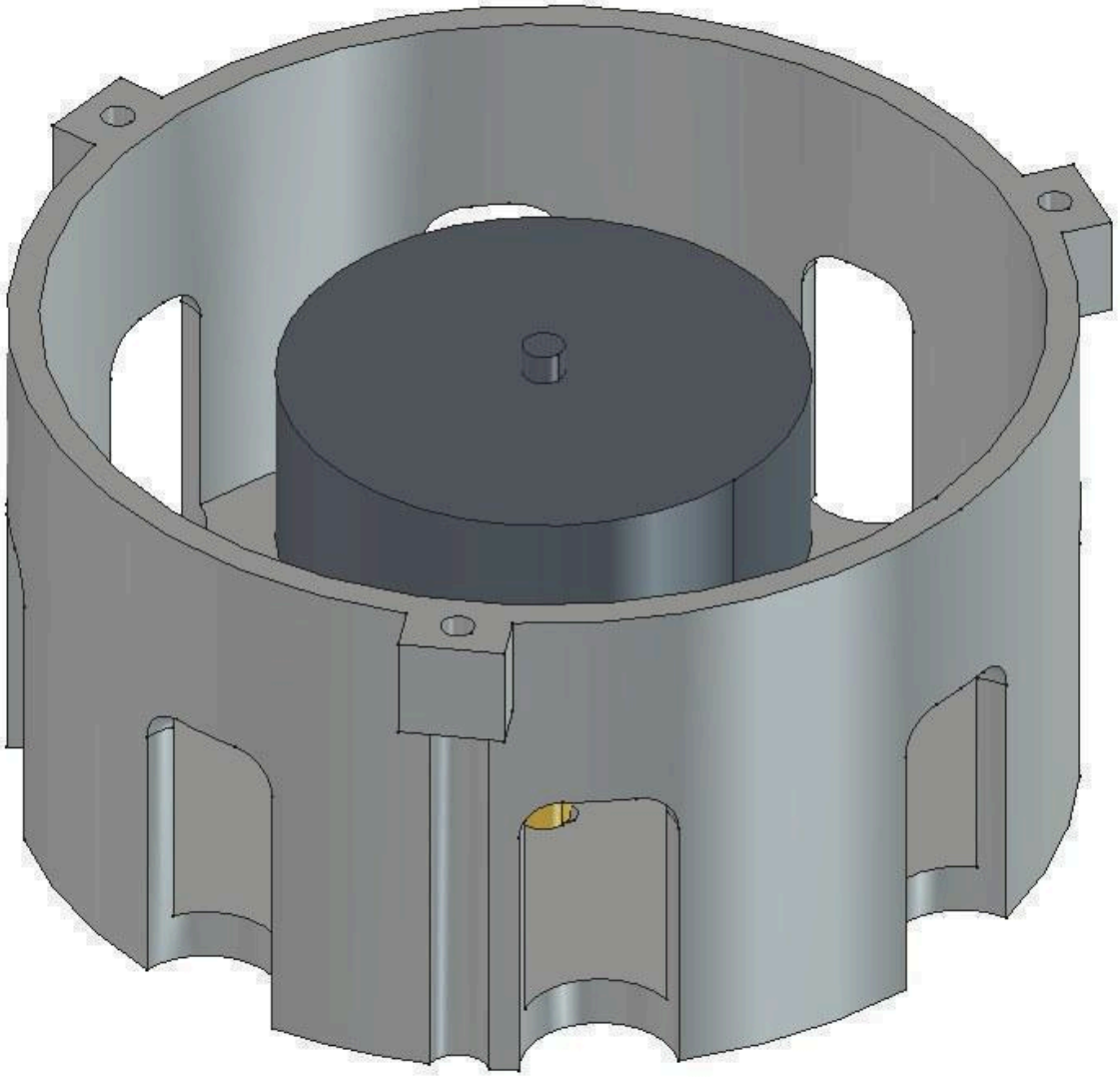


Figure 7. Motor (dark gray) installed in the base.

Step 2. Fit the rotor adapter

1. Place the adapter on the top of the rotor. The motor shaft stub fits into the small recess underneath the adapter.
2. Line up the 4 holes with the rotor's 4 threaded holes (12 mm circle) and fit 4 × M3 × 8 socket head screws. The heads sit flush in 3 mm counterbores. The screws pass through 4.5 mm of adapter and go 3.5 mm into the rotor. The rotor's threads are only 5 mm deep, so never use longer screws: they could reach the motor windings.
3. Spin the motor by hand. The adapter must run true, with no wobble.

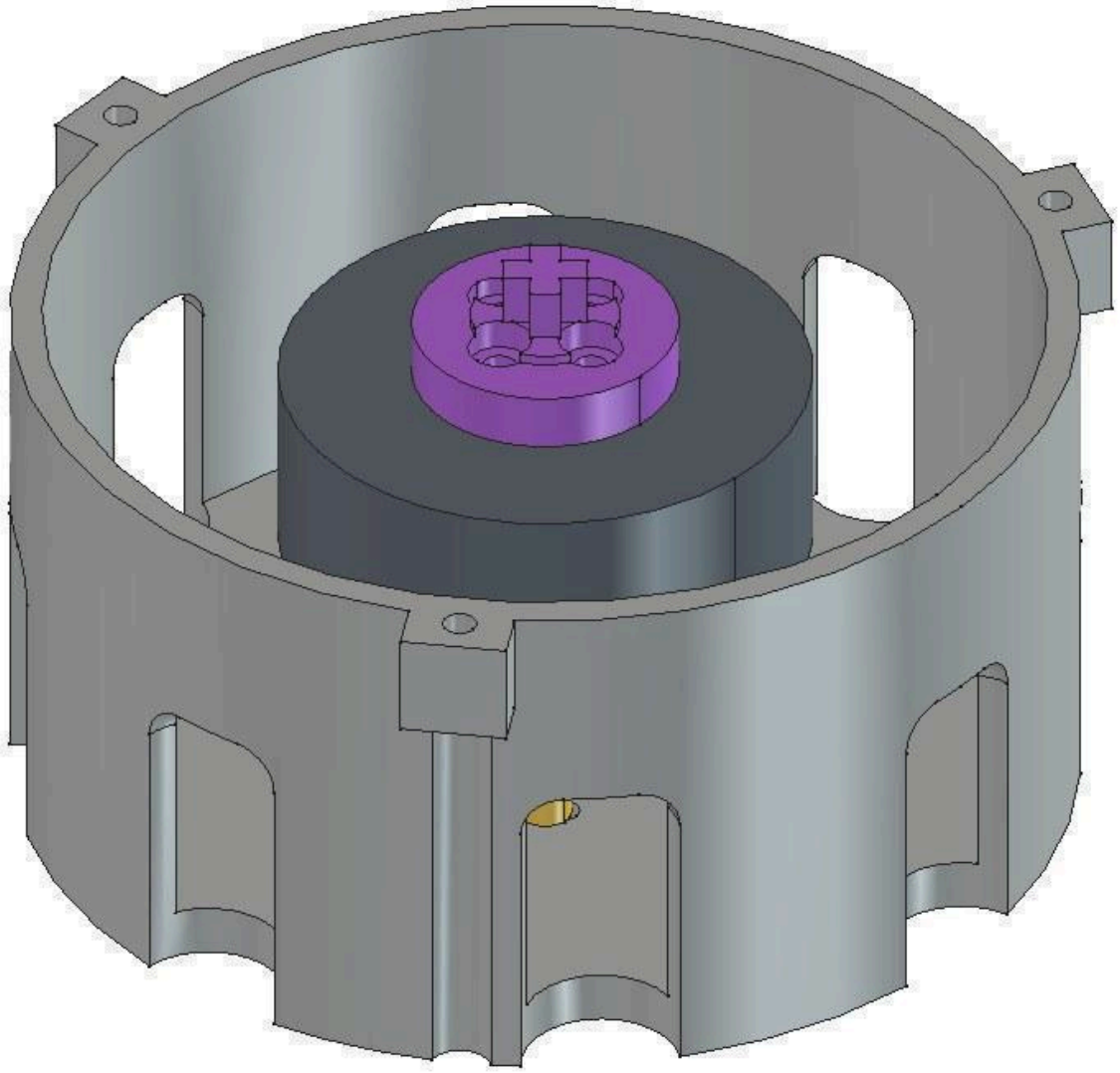


Figure 8. Rotor adapter (purple) screwed to the rotor. The cross key will drive the sun gear.

Step 3. Fit the output bearing into the ring

1. Stand the ring on its gear end, top up.
2. Drop the 61812 bearing in from the top until it sits flat on the shoulder.
3. Put the bearing retainer on top, line up its 6 holes with the ring's holes, and fit 6 × M3 × 10 screws. Tighten evenly in a criss-cross order until the bearing's outer ring is clamped. Do not over-tighten: the threads are cut into plastic.

Step 4. Build the planet sub-assembly

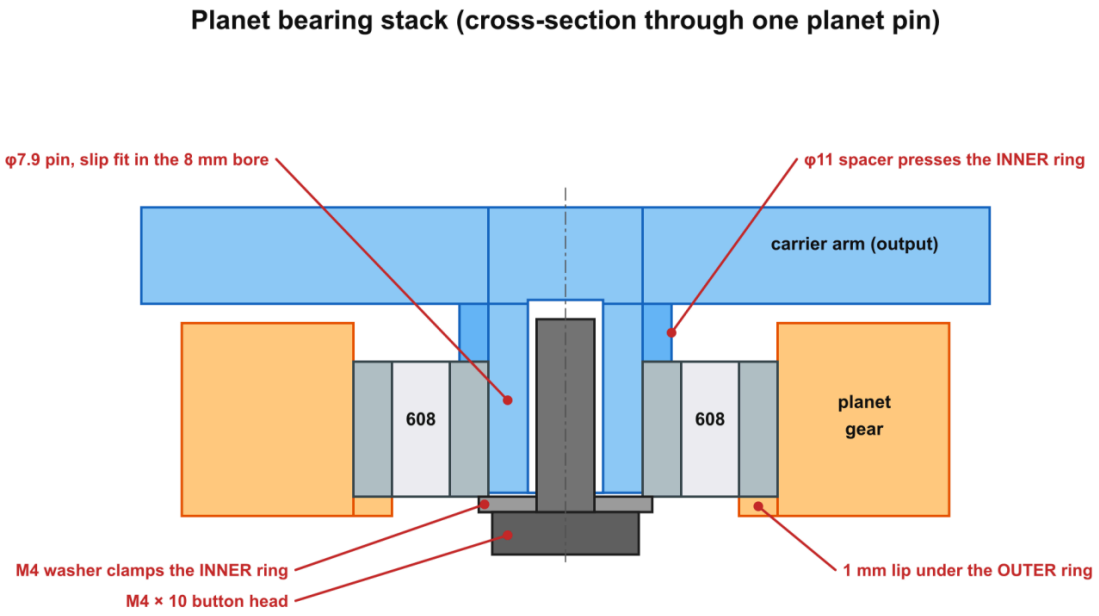


Figure 9. How each planet is held. The screw clamps only the bearing's inner ring, so the planet spins freely on the outer ring.

1. Press one 608 bearing into the top of each planet until it sits on the 1 mm lip. Push on the outer ring only.
2. Hold the carrier pins up and put the MR105 bearing into the center pocket.
3. Slide a planet onto each pin with the lip side facing away from the carrier.
4. From the lip side, fit an M4 washer and an M4 × 10 button head into the pin's pilot hole (φ3.9, 10 mm deep; it prints at about 3.8 mm). Tighten until the planet spins freely with no wobble. Do not over-tighten: the thread is cut into plastic.
5. Drop the sun gear into the middle so its pin goes into the MR105. The cross-shaped pocket must face up. Turn it slightly so its teeth mesh with all three planets.

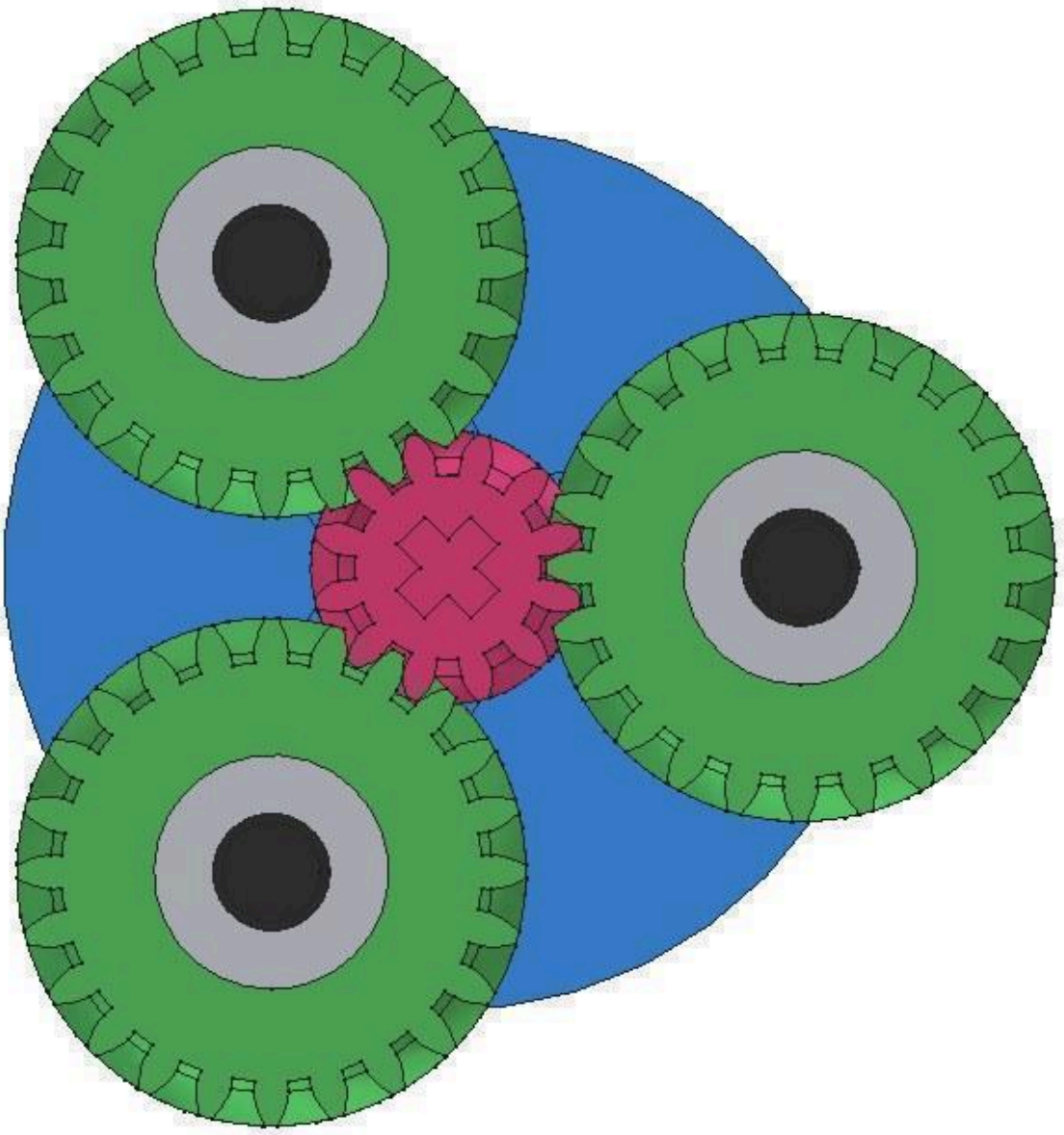


Figure 10. Carrier sub-assembly seen from the planet side: three planets on 608 bearings, screws in the centers, sun in the middle.

Step 5. Put the carrier into the ring

1. Turn the ring over so the retainer is on the table and the gear end faces up.
2. Lower the carrier sub-assembly into it, output hub first, so the hub goes into the bearing.
3. The teeth are helical, so twist the carrier gently back and forth while pushing. This lets the planets thread into the ring teeth.
4. Push until the carrier's rim sits against the bearing's inner ring. The hub ends 0.2 mm below the bearing's top face.

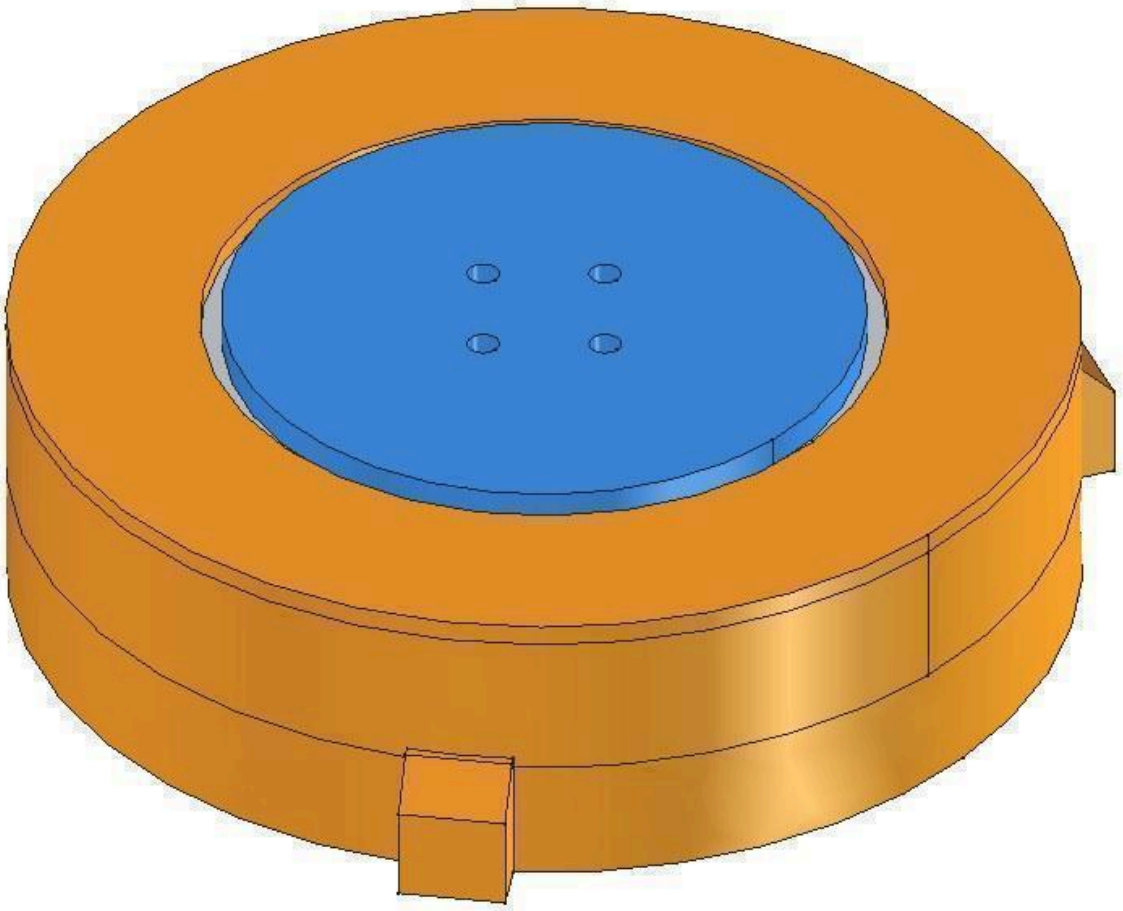


Figure 11. Carrier and planets inside the ring gear (V1; in V2 the bearing is already in the ring).

Step 6. Join the gearbox to the base

1. Turn the gearbox over so the planets face down and lower it onto the base.
2. Rotate the output slightly until the sun's cross pocket drops onto the adapter's cross key.
3. Line up the three ring tabs directly above the three base tabs. The ring's bottom sits directly on the base, with no gap.
4. Insert 3 × M3 × 16 screws from below through the base tabs into the ring tabs, and tighten evenly.
5. Turn the output by hand. It should rotate smoothly all the way round, and it should be possible to back-drive the motor.

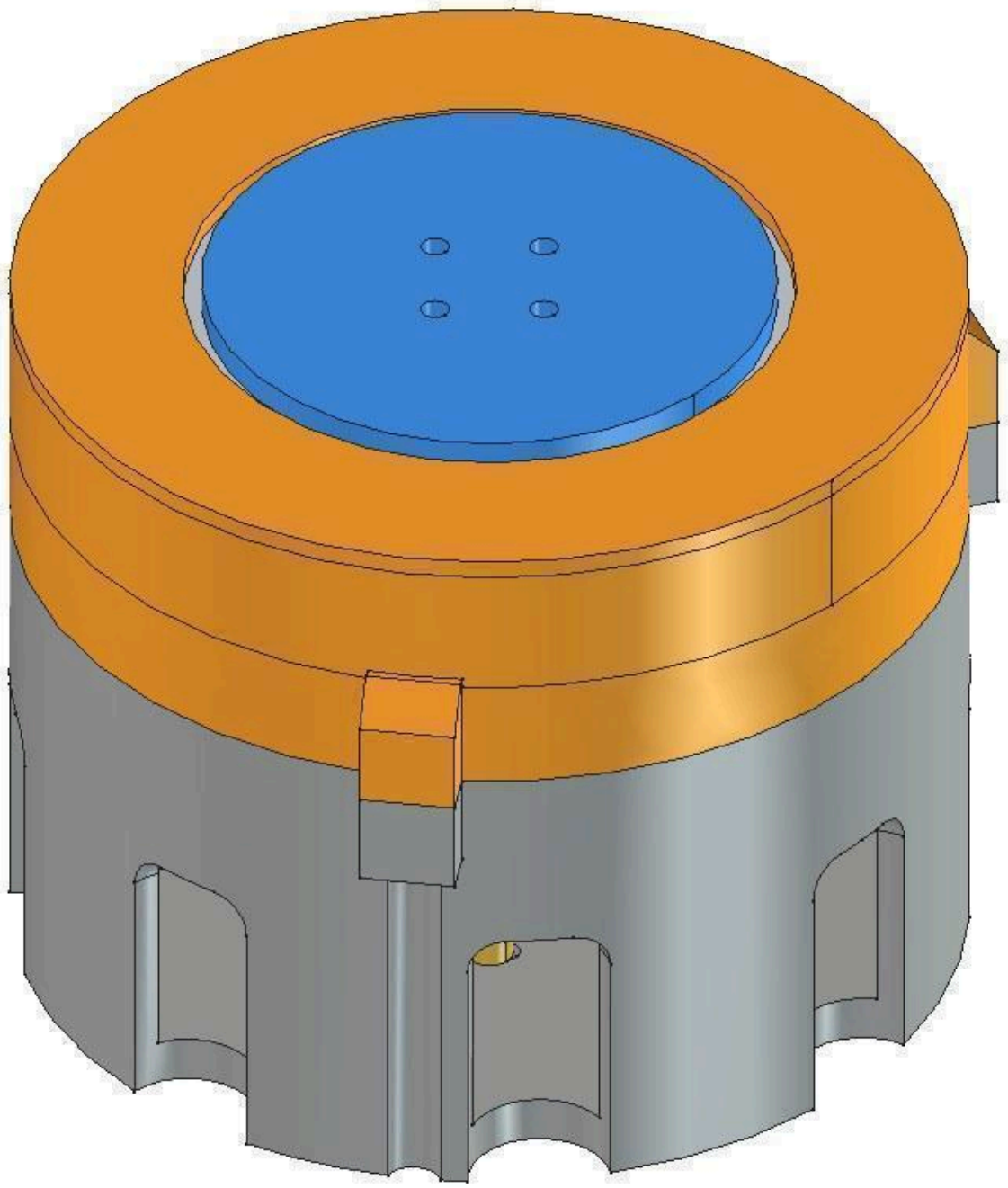


Figure 12. Finished mechanical assembly (V1, before the output flange).

Step 7. Fit the output flange

1. Place the output flange on the carrier hub, counterbores up, and line up its 6 holes with the carrier's holes.
2. Fit 6 × M3 × 12 screws and tighten evenly in a criss-cross order. The flange presses on the bearing's inner ring, not on the hub.
3. Turn the output by hand again. It should still turn smoothly, with no wobble or axial play when you push and pull on the flange.

4. The arm, or any other link, bolts to the flange's 4 M3 holes on a 24 mm circle.

Step 8. Fit the arm and the scale pusher (optional)

Arm A1 is a 100 mm lever for measuring the joint's real output torque (section 12). Print PlanetaryGearBox 5010-Arm A1.3mf flat, counterbores up; it needs no support. The **scale pusher** (PlanetaryGearBox 5010-Arm A1 Pusher.stl) also prints flat with no support.

1. Place Arm A1 on the output flange, counterbores up, and line up the 4 holes at its hub with the flange's 4 M3 holes. It fits in four positions, 90° apart.
2. Fit 4 × M3 × 8 screws. The heads sit in 3.2 mm counterbores, and about 5 mm of thread goes into the flange. Do not use M3 × 10: they would reach 7.2 mm into the 7 mm deep flange holes.
3. For torque calibration, fit the pusher under the arm's far end. Line up its 4 holes with the arm's 4 counterbored holes at 100 mm and fit 4 × M3 × 8 from the top. They thread into the pusher's 2.9 mm holes, the same size as the flange's. The pattern lets the finger point to either side: point it toward where the scale will be.
4. Turn the output a full turn by hand. The arm and the pusher clear the housing all the way round.

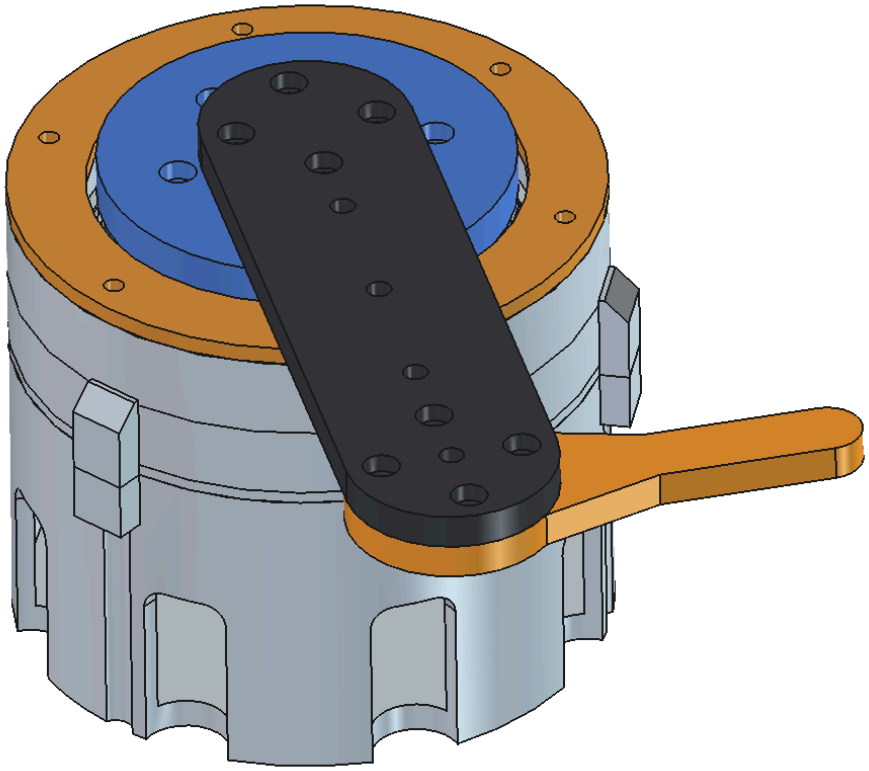


Figure 12b. Arm A1 (black) and the scale pusher (orange) on the output flange.

Lever length	100 mm from the axis to the far hole pattern and to the pusher's contact point, 118 mm overall
Section	36 mm wide × 6 mm thick, straight sides with round ends
Holes	The flange pattern (4 × M3 on a 24 mm circle) at both ends; Ø4.2 mm load holes at 25, 50 and 75 mm
Pusher	6 mm plate under the tip with a finger reaching 70 mm sideways and a rounded end. The contact point is exactly 100 mm from the axis, however far the finger reaches.
Mass	Arm about 15 g printed (0.2 mm layers, 15% infill), 33 g if solid; pusher about 13 g if solid

The first calibration arm (PlanetaryGearBox 5010-Arm.3mf: 12 × 8 mm, tapered, with tip nubs) still fits the same way, with 4 × M3 × 10.

5. Wiring

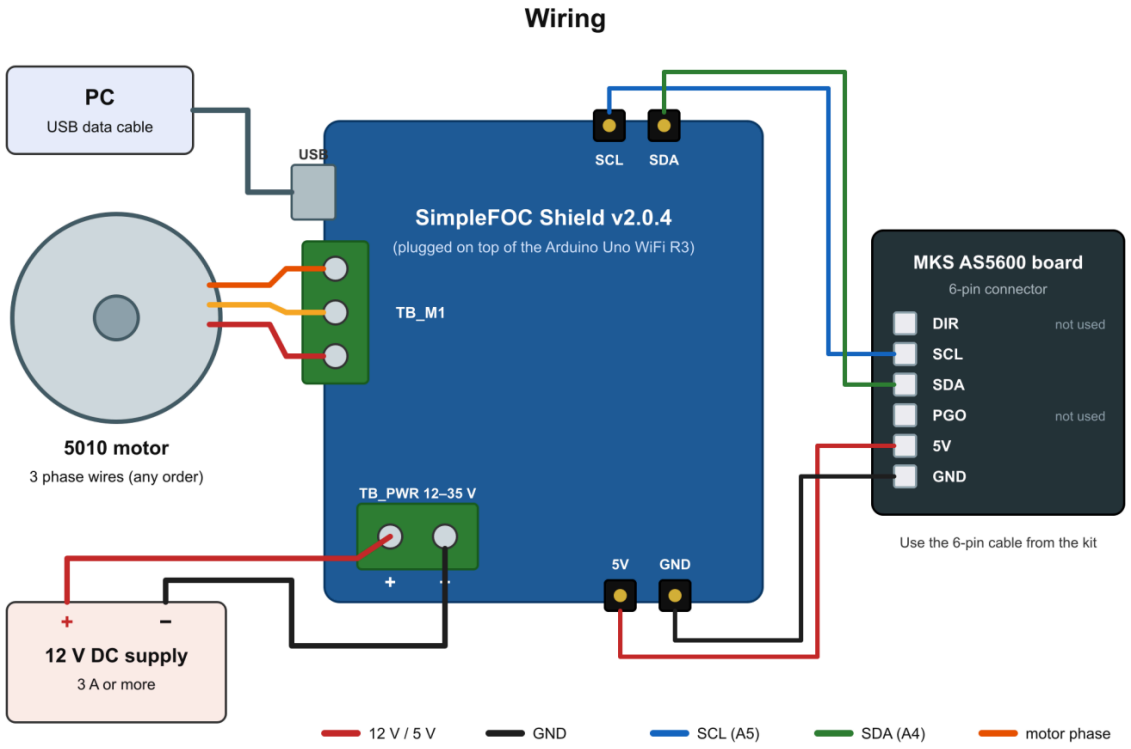


Figure 13. Wiring diagram. It shows the original Uno; the connections to the shield are the same on the Mega.

1. Plug the SimpleFOC Shield onto the Arduino Mega. It sits on the Uno-sized header at the USB end of the board.
2. Connect the three motor phase wires to the shield's **TB_M1** terminal. If you change the order of the wires, recalibrate (section 7).
3. Connect the encoder to the shield as shown in the diagram:
    - SCL to SCL
    - SDA to SDA
    - 5V to 5V
    - GND to GND
    - Leave DIR and PGO unconnected.
4. Use the shield's **SDA/SCL header**, not A4/A5. On the Mega, I2C is on pins 20/21, which the SDA/SCL header reaches; A4/A5 are ordinary analog pins.
5. Connect the 12 V supply to **TB_PWR**. Check the polarity. Do not switch it on yet.
6. Connect the Mega to the PC with the USB cable.

The shield must use its default pin settings: PWM on pins 9, 5 and 6, enable on pin 8. These are set by solder pads on the back. Leave them as they come from the factory. The current sensors are on A0 and A2 and need no wiring.

6. Software setup

1. Install the **Arduino IDE** (version 2), or Arduino CLI.
2. In the Library Manager, install **Simple FOC** (version 2.4.0 or newer).

3. Plug in the Mega. Windows should show it as an Arduino Mega 2560 COM port (on the build PC it is **COM5**; the number can change between USB ports).
4. Open the firmware **QDD5010.ino** from the project's Code/QDD5010 folder.
5. Select board **Arduino Mega or Mega 2560** and its COM port, then upload. With Arduino CLI in PowerShell: **& "C:\Program Files\Arduino CLI\arduino-cli.exe" compile --upload -p COM5 --fqbn arduino:avr:mega QDD5010**
6. Open the Serial Monitor at **115200 baud** with line ending set to **Newline**. With Arduino CLI: **& "C:\Program Files\Arduino CLI\arduino-cli.exe" monitor -p COM5 -c baudrate=115200**. Close the monitor before uploading again, because it holds the port.

What the firmware does

- **Built on SimpleFOC:** it uses the open-source SimpleFOC library and runs the FOC loop at about 380 Hz on the Mega. The program uses 18% of the Mega's flash.
- **Current from measured motor data:** it sets the motor voltage from the measured resistance (0.69 Ω, including the driver) and back-EMF (0.0113 V per rad/s), so the real current matches what the controller asks for. This is called feedforward. The current limit is **3.5 A**, and a hard **3.0 V** cap on the motor voltage acts as a backstop.
- **Current sensing for monitoring:** it reads the shield's current sensors and shows the real current in the telemetry (Iq meas). It does not correct the current from these readings by default: the sensors read about ±0.3 A of noise, and chasing that noise made the joint jitter about 7° at the motor while holding. A correction loop can be switched on with the Q command.
- **Saved calibration:** it stores the motor's calibration (sensor direction CCW, electrical zero 1.0063) and the current-sensor mapping, so start-up is quick and the same every time.
- **Starts in angle mode:** on power-up it holds the position the motor is in.
- **Output angle and speed:** it knows the 6:1 ratio, so you can command and read the output angle and speed directly.
- **Telemetry:** off at start-up so you can type commands. Type M to print loop rate, angles, speed, currents and voltages 20 times per second, without slowing the control loop.
- **Commands:** upper or lower case both work. Type H or ? for the command list.

7. First power-up and calibration

1. **Clamp the actuator** to the bench before applying power. Never hold it by hand while it runs.
2. Switch on the 12 V supply, then press reset on the Arduino. Opening the Serial Monitor also resets it. The board waits 2 seconds before it starts, so the monitor catches the whole start-up text.
3. Watch the Serial Monitor. A good start looks like this, ending with the command list and the gains. Then the screen stays quiet until you type a command:
    - MOT: Init
    - MOT: Enable driver.
    - MOT: Align sensor.

    - Skip dir calib.
    - Skip offset calib.
    - MOT: Align current sense.
    - MOT: Success: 1
    - MOT: Ready.
4. Type **B15** and press Enter. The output turns 15° and holds. Type **B0** to return.

Recalibrating

Recalibrate after you change the motor, re-glue the encoder magnet, remount the motor or change the motor wiring order. Calibrate with the motor fixed in the base and the sun, planets and carrier removed, so friction cannot spoil the measurement:

1. Disable the two saved calibration lines in the firmware's setup section. They are marked "Calibrated".
2. In the same section, change the align voltage line from CS_ALIGN_VOLTAGE back to ALIGN_VOLTAGE. Calibration needs 1.0 V to move the rotor.
3. Upload and power up. The motor makes a short slow movement while it calibrates, then prints "sensor dir" and "Zero elec. angle".
4. Copy those two values back into the firmware, re-enable the lines, set the align voltage back to CS_ALIGN_VOLTAGE and upload again.

Current-sensor mapping: the firmware fixes which sensor reads which phase (A2 is phase A with its sign inverted, A0 is phase B), instead of letting SimpleFOC test it at every start-up. That automatic test gave a different answer on different boots, and the wrong answers made the motor swing or spin. If you change the shield's pin settings, the mapping has to be found again.

8. Serial commands

Type a letter followed by a number, then press Enter. Upper and lower case both work (R30 or r30). Type **H** or **?** to print the list on the board; it groups the commands into angle, speed, torque, power, limits, tuning and info. **B** and **S** work at the output; **A** and **R** work at the motor, which turns 6 times as far and as fast. All amps are real amps. Settings you change last until the next reset.

Quick reference

To do this	Type
Move the output to 30°, then back	B30, then B0
Set the current position as zero	Z

Spin the output at 10 RPM, then stop	S10, then S0
Push gently, then let go	T0.5, then T0
Stop now (output goes limp)	E0
Turn back on	E1, then a B command
Watch the readings, then stop them	M, then M
Allow more torque (both caps)	QV2, then C2.5
See the settings	G
See all commands	H or ?

All commands

Command	What it does	Example
B<deg>	Move the output to an angle and hold it (angle mode)	B30, B-15, B0
Z	Make the current position the zero (motor and output)	Z
A<deg>	Move the motor to an angle and hold it (angle mode)	A90, A0

S<rpm>	Spin the output at a speed (velocity mode); S0 stops	S10, S-5, S0
R<rpm>	Spin the motor at a speed (velocity mode); R0 stops	R60, R-30, R0
T<amps>	Apply a torque as a motor current (torque mode)	T0.5
C<amps>	Set the current cap (default 2, max 4)	C1.5
V<rad/s>	Motor speed limit used in angle mode (default 20)	V10
P / I / D<value>	Velocity loop gains (default 0.1 / 1 / 0)	P0.08, I0.8
N<value>	Angle loop gain (default 15)	N10
F<seconds>	Velocity filter time constant (default 0.02)	F0.01
O<A/s>	Velocity loop output ramp (default 1000)	O1000
Q + letter + value	Current loop settings:	QV2, QP0.2, QX0.69, QL2
K<kv>	Back-EMF compensation KV for the estimated mode (QL0) only; 0 turns it off (default 360)	K0
E1 / E0	Enable / disable the driver (E0 is the quick stop)	E0
M	Telemetry on/off (off at start-up)	M

G	Print all gains, limits and calibration	G
H or **?**	Print the command list	H

Typing while telemetry scrolls: the monitor still sends what you type, even though the text is buried in the output. Type **M** and press Enter to stop the telemetry.

The zero is not kept at power-off. The encoder only knows the motor's angle within one motor turn, which is 60° at the output. After every power-up, move the output to your reference position and type **Z**.

Telemetry line fields:

- **Hz:** control loop rate
- **Angle:** motor angle in degrees
- **Vel:** motor speed in RPM (divide by 6 for the output)
- **Iq set:** the current the controller asks for, in amps
- **Iq meas:** the measured torque-producing current, in amps. In the estimated mode (QL0) this is only a filtered copy of Iq set, not a measurement.
- **Id:** the measured sideways current, in amps. It should stay near 0.
- **Uq / Ud:** the voltages applied to the motor, in volts
- **Out:** output angle in degrees (motor angle ÷ 6)

Web console

The web console (**Code/WebApp/index.html**) does everything the Serial Monitor does, and adds live charts, a 3D view of the joint, a force readout, contact detection and the torque calibration. It talks to the Mega straight from the browser (Web Serial), so it needs Chrome or Edge on a desktop. It reads the same telemetry the M command prints.

1. Close the Arduino Serial Monitor. Only one program can use the port.
2. Run start.bat in Code/WebApp. It serves the page at http://localhost:8765 and opens it. (Opening index.html directly also works in some browsers.)
3. Click Connect and choose the Mega's port (COM5). The board resets, and the console turns the telemetry on by itself about 3 s later.
4. To try the console without hardware, click Simulator instead. It models the joint, including the V2 gearbox's tight zone and a virtual wall at +75° of output angle. Type W in the Log line to switch the wall off or on.



Figure 14. The web console pushing 2 A against the simulator's wall: 0.112 N·m, 114 g at 100 mm.

The page has three columns. Every card, the 3D view and both charts have a small **i** button that opens a short explanation of that part.

Part	What it does
Top bar	Connect (Web Serial) or Simulator; the link, loop rate and mode; Enable (E1, then holds where the arm is) and STOP (E0, or the Esc key).
Drive	Three tabs. Angle: a number and Go, buttons from −90° to 90°, and Set zero. Speed: output RPM, Spin and Stop. Torque: amps, Push and Release, with the expected force on a load from the calibration. Enter in a box sends it.
Force	FREE or CONTACT, the outside torque in N·m and in grams at the lever length, and a list of contacts with time, angle and size.
3D view	The actuator with Arm A1 and the pusher at the measured angle, also when the motor is off and the arm is turned by hand. A faint blue arm shows the angle

	target; a red arrow at the pusher tip shows an outside push. "No data" means telemetry stopped: type M in the Log line.
Charts	The last 20 s: output angle and target; current set and measured with the contact band, and the outside torque on the right axis.
Log tab	Every command and reply, and a line for any firmware command (H lists them, G shows gains and limits). Telemetry lines are hidden unless ticked.
Setup tab	Limits (C and QV; power-up 3.5 A and 3.0 V), Calibration values (Kt, push friction, lever), Contact detection (threshold, Learn free motion), the Torque calibration table, and Display and recording (flip the 3D direction, axis horizontal, Record CSV).

Force and contact. Held against a load (torque mode, or angle mode at rest), the outside torque is Kt × (I − push friction), with the values from the scale calibration in section 12: 0.099 N·m per amp and 0.87 A, good to about ±8 g at 100 mm between 1 and 3.5 A. While moving, the console predicts the current that free motion needs (friction plus the current that speeds the arm up or slows it down), and treats current above that as outside load: Kt × the extra current. Holding still, current up to the friction level is ignored, because friction alone can hold that much.

- Learn free motion (Setup tab) teaches it this joint's friction. It turns the output a full turn each way at 5 RPM, then makes ±40° moves, and builds a friction map every 10° of output angle in each direction (which captures the tight zone) plus the speed-up term. It also sets the threshold from the noise it saw (0.2 A at least). The arm must be free to swing all the way round. The model is saved in the browser; run it again after mechanical changes.
- Threshold is the extra current that counts as contact. Raise it if contact shows up during normal moves; lower it to feel lighter touches.
- In torque mode (T), contact means the arm stopped while pushing harder than friction.

Torque calibration table (Setup tab): it follows section 12. The rows go from 0.5 to 3.5 A, increasing only. Apply sends that row's current (tick the negative box if the arm moves away from the scale). After the reading settles, type the grams and click Save: it stores the average measured current of the last second next to the reading. From two or more rows at 1 A or above it fits Kt and the push friction, and Use fit copies them into Calibration values. It is prefilled with the November 8 results.

9. Tuning

The firmware ships with gains tuned on October 30 with the gearbox fitted:

Setting	Value	Notes
Feedforward	0.69 Ω, 0.0113 V per rad/s	Measured at 30–120 RPM. This sets the motor voltage.
Current correction loop P / I	0 / 0 (off)	With it on (0.2 / 100), sensor noise made the hold jitter about 7° at the motor. See the rule below if you turn it on.
Current filter	0.001 s	Only matters when the correction loop is on
Velocity P / I	0.1 / 1	0.15 / 1.5 doubles the holding jitter
Velocity filter	0.02 s	Smooths the noisy speed reading; 0.01 jittered more
Velocity output ramp	1000 A/s	200 was too slow to reverse the current when braking and caused large swings after big moves
Angle P	15	
Current limit	2 A	Real amps

Results with the gearbox: 60° moves at the output settle in about 0.3 s with no overshoot and stop within 0.2°. Holding still, the motor moves about 0.5° (under 0.1° at the output). S10 runs the output at 9.8 RPM.

Check the mechanics before retuning. If the joint suddenly jitters, stalls or feels weak, type R30, then M, and watch Iq meas. A free-turning gearbox needs about 0.3 A. Much more than that means something is dragging: a loose planet screw, a rubbing motor bell or something touching the output. Type R0 to stop.

Why the velocity gains look smaller than before: with the Uno, the firmware used estimated amps based on the motor's 0.12 Ω alone. The real resistance with the driver is

0.69 Ω, so one estimated amp was only about 0.17 real amps. The new 0.1 / 1 is the old 0.5 / 8 converted to real amps, and the old 6 A estimated limit was only about 1 A of real current.

Current loop rule (only if you turn the correction loop on): the loop runs about 380 times per second, and the motor's inductance is far too small to slow the current down between updates. The current loop's gain per update, roughly P × 1.45 + I × 1.45 ÷ 380, must stay well below 1, or the current itself oscillates.

To re-tune:

1. **Clamp the actuator.** Tuning a loose or hand-held motor gives wrong results and is unsafe.
2. **Change one gain at a time** and test with B30 and B0 after each change.
3. **Back off at the first sign of trouble.** Buzzing, overshoot or oscillation means you have gone too far; go back to the last good value. Type E0 to stop at once.
4. **Save the result.** Record the final values with G and copy them into the firmware.

10. Troubleshooting

Symptom	Likely cause	Fix
No COM port appears	Charge-only USB cable	Use a data cable and check Device Manager
Upload fails ("not in sync" or port busy)	Wrong COM port (it can change); Serial Monitor holding the port; wrong board selected	Check the port in Device Manager, close the Serial Monitor, select Arduino Mega 2560 (fqbn arduino:avr:mega), retry
"Unexpected token" when running arduino-cli in PowerShell	A quoted program path needs the call operator	Put **&** in front of the quoted path
Top of the start-up text or command list is missing	The monitor connected late	Type H to print the command list again

Can't type in the Serial Monitor	Telemetry is on (M) and scrolls 20 times per second	Type M and press Enter (even if you can't see it) to pause telemetry
Commands do nothing	Serial Monitor line ending is not Newline	Set the line ending to Newline
Motor jitters while holding	Something dragging in the gearbox; current correction loop on	Check the slow-spin current (section 9); type QP0 then QI0
"Failed to notice movement" at start-up	12 V supply off; loose motor phase wire; calibration voltage too low	Switch on 12 V, check TB_M1 screws, then recalibrate
Angle stays at 0.00 or never changes	Encoder not detected: wired to A4/A5 instead of SDA/SCL, SDA/SCL swapped, no 5 V, magnet missing	Use the shield's SDA/SCL header (Figure 13); the encoder should answer at I2C address 0x36
"Current sense init FAILED" or "Init FOC fail" at start-up	Shield not fully seated; 12 V off	Reseat the shield, switch on 12 V, reset
Motor does not move, or only twitches	Current limit too low; bell rubbing the windings	Keep C at 1.5 or more; spin the motor by hand with the power off and check for scraping
Oscillation or large swings after a move	Feedforward off; gains too high; actuator not clamped	Type E0, check QX is 0.69 (G prints it), lower P, I or N, clamp the actuator, then try again

Iq meas does not follow Iq set	Voltage cap reached (high speed or load); motor warm (its resistance rises with temperature); wrong current-sensor mapping	Lower the speed or load; if it happens at rest, check the shield's pin settings (section 7)
Output angle is wrong after power-up	The zero is not saved	Move to the reference position and type Z
Driver or motor gets hot	Current at its limit for long periods; gains fighting each other	Lower C, reduce gains, check that nothing in the gearbox binds
Garbled serial text	Wrong baud rate	Set the Serial Monitor to 115200 baud with Newline line ending
Gearbox tight or clicking	Print blobs or elephant foot on teeth; bearings not fully seated; wrong sun height	Clean the teeth, check the test fits, reseat the bearings, use the +2 mm sun
Planet wobbles	M4 screw loose or bearing not fully pressed in	Press the bearing onto the lip, then tighten the screw until just snug

11. Safety

- **Low resistance:** the motor plus driver measure only 0.69 Ω, so a few volts can drive several amps. Do not remove the firmware's current limit or voltage cap.
- **Clamp the actuator** for every motion test, and keep fingers away from the gears and rotor.
- **Quick stop:** type E0 to stop instantly, or switch off the 12 V supply.
- **Driver limits:** the L6234 driver handles about 5 A peak but heats up beyond a few amps continuous, because the shield has no heatsink. Touch-check its temperature when testing at higher current.
- **Sensor range:** the shield's current sensors read about ±5 A. Above that the readings clip and current control stops working.

12. Torque limits and upgrades

The motor could deliver far more torque. Today the limits are, from tightest to loosest:

1. Current limit (3.5 A, measured amps): the firmware's power-up setting since November 8.
2. Voltage cap (3.0 V): sized for the current limit. 3.5 A needs about 2.4 V on a cold winding and up to 3.0 V once it is warm (0.69 Ω cold, about 0.85 Ω seen at 4 A), plus back-EMF while moving.
3. The L6234 driver: about 5 A peak, overheats at a few amps continuous, and most of the 0.69 Ω is in its switches, which wastes power as heat.
4. The shield's current sensors: about ±5 A.
5. The motor: about 0.023 Nm per amp, or 0.138 Nm per amp at the output in theory. Measured at the output: 0.099 Nm per amp above 0.87 A of gearbox friction (Calibrating torque, below). It is not the bottleneck.

At the 3.5 A default the joint pushes about 0.26 N·m at the output, which is about 260 g at 100 mm. Current sensing has been in use since October 30: Iq meas shows the real torque-producing current, and the current limit stays on the safe side as the motor warms, because the resistance rises.

Setting the caps

Two caps limit torque, and both can be changed from the Serial Monitor or the web console's Setup tab:

Cap	Command	Default	Highest allowed
Current	C<amps>	3.5 A	4 A
Voltage	QV<volts>	3 V	3 V

- Change both together. The current cap only helps if the voltage cap allows that much current: about 0.7 V per amp on a cold winding, 0.85 V warm, plus a little for speed. Lowering the current cap on its own is always safe.
- Watch the heat: type M and watch Iq meas while the joint works, and touch the L6234 chip on the shield. It should be warm, not hot. Pushes near 3.5 A for more than a few seconds heat the winding and the driver, and the real current then falls below the setpoint: with 4 A set, a warm motor drew only about 3.3 A.
- Values above the maximum are reduced to it (QV9 sets 3 V, C10 sets 4 A). The ceilings keep the shield inside its ±5 A sensing range: 3 V across 0.69 Ω is about 4.3 A at a stall. Type G to see the caps (Ilim and Vcap).

- They reset at power-up to 3.5 A and 3.0 V. To keep other values, change the defaults in the firmware's setup section.

Upgrade path:

- Small step: C4 for short pushes. The voltage cap is already at its 3 V maximum.
- A dedicated FOC driver with current sensing, such as the B-G431B-ESC1: low-resistance switches, a much faster current loop and roughly 5–10× today's torque. This removes the shield's ceiling.
- Arduino UNO Q: much faster (STM32U585, 160 MHz), but SimpleFOC does not officially support it yet.

Calibrating torque

The firmware only knows the motor current. To turn current into output torque, the pusher on Arm A1 presses a kitchen scale 100 mm from the axis while the current is stepped up.

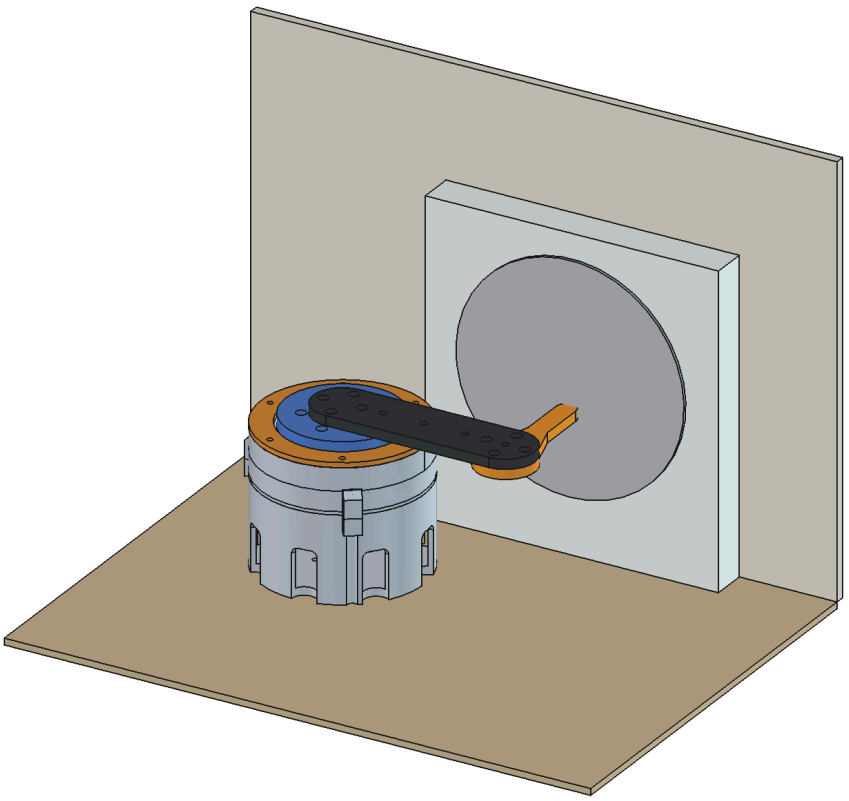


Figure 15. The calibration rig: joint clamped flat, scale standing on its edge against a wall, pusher on the pan.

1. Clamp the actuator flat on the table, output up, so the arm swings level and its own weight does not load the scale. The clamp must stop the base from **twisting**, not just lifting: one clamp can act as a pivot. Use two on opposite sides, or one clamp and a stop block. In the first try the base turned about 30° in a single clamp, and the readings came out at a third of the real torque.
2. Fit Arm A1 and the pusher (section 4, step 8), with the finger toward the wall.

3. Stand a kitchen scale (1 g resolution) on its edge against the wall, pan facing the joint. The contact point is 86 mm above the table, on the lower half of most pans.
4. With the motor off (STOP or E0), turn the arm until the pusher just touches the pan, with the arm parallel to the wall. Each degree off parallel makes the lever about 1% wrong. Tare the scale.
5. Type T0, then E1, then step up: T0.5, T0.8, T1.1, T1.4, T1.7, T2.0, T2.5, T3.0, T3.5. Wait a second at each step and write down the grams. The web console's Setup → Torque calibration table does the same with Apply and Save.
6. **Only ever increase the current**, and never let the arm back off the scale between steps. Going only upward, friction always works against the push, so a straight-line fit gives both the torque per amp and the friction. If a positive current moves the arm away from the scale, use negative values.
7. Finish with T0, then E0. Output torque (N·m) = grams ÷ 1000 × 9.81 × 0.100.

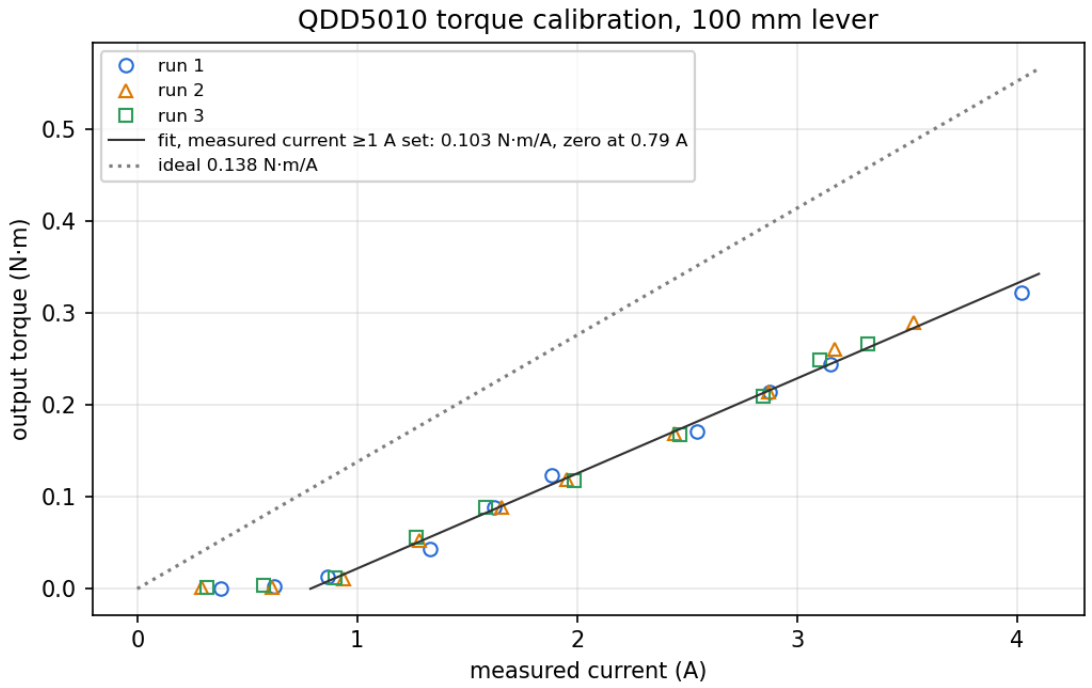


Figure 16. Three runs on November 8, 2026, plotted against the measured current.

Set current (A)	Run 1 (g)	Run 2 (g)	Run 3 (g)	Torque, mean (N·m)
0.5	0	1	2	0.001
0.8	3	2	4	0.003

1.1	13	11	12	0.012
1.4	44	54	57	0.051
1.7	90	90	90	0.088
2.0	126	121	120	0.120
2.5	175	172	171	0.169
3.0	218	218	214	0.213
3.5	249	266	254	0.251
4.0	328	295	272	0.293

Results (November 8, 2026):

- Above the dead zone, output torque ≈ 0.099 × (I set − 0.87 A) N·m between 1 and 3.5 A, within about ±8 g at 100 mm. Against the measured current: 0.103 × (I − 0.79 A), good up to 4 A within about ±7 g.
- About 75% of the ideal 0.138 N·m per amp (360 KV × 6:1) reaches the output, which is normal for a printed planetary gearbox.
- Below about 0.9 A almost nothing reaches the scale: that current goes into gearbox friction under load. It is much more than free motion needs (about 0.35 A), because the friction grows with the load on the teeth and bearings.
- The runs agree within 6 g up to 3 A, except 13 g at 1.4 A, just above the dead zone. At 4 A the reading depends on temperature: from cold the driver reached 4.02 A, warm only 3.3 A. Against the measured current, all three 4 A points sit on the line.
- The web console uses 0.099 N·m per amp and 0.87 A (Setup → Calibration values) for its torque hint and force readout.
- Tight zone: the V2 gearbox needs more current between about 90° and 180° of output angle. These runs were at about 14°. To map it, repeat the test with the arm bolted on at another of its four positions.

- Back-driven loads: the model is for the motor pushing into a load. When an outside force drives the joint backwards, friction acts the other way and the real torque is higher than shown.

13. Design notes

Gears	Module 1.5, 20° helical. Sun 12, planets 24, ring 60 teeth. 10 mm planet face width, 12.5 mm sun. 0.1 mm backlash per mesh.
Planet spacing	27 mm from center, 3 planets at 120°
Motor	MKS 5010, 360 KV, 7 pole pairs, 0.12 Ω winding (0.69 Ω measured with the driver), 50 µH
Encoder	AS5600, 12-bit, I2C at 400 kHz
Current sensing	INA240A2 on the shield, 0.01 Ω shunts, gain 50, about ±5 A, on A0 and A2
Control	SimpleFOC 2.4 on an Arduino Mega, angle → velocity → current cascade. Voltage from resistance and back-EMF feedforward; measured d/q current for monitoring. About 380 Hz.

- **6:1 ratio:** a good balance between torque and back-drivability, and it kept the ring the same outside size as the earlier 2804 version.
- **608 planet bearings on 8 mm pins clamped by screws:** this replaced the earlier thin press-fit pins, which broke.
- **Helical teeth:** they run smoother and quieter, at the cost of a small axial push on the planets. The clamped bearings take that push.
- **Motor mounted on the encoder kit's own legs:** no extra brackets are needed.
- **Output bearing clamped on all four faces (V2):** in V1 the bearing only resisted the output being pulled away from the motor. Pushing toward the motor, or tipping loads from an arm, went through the planets and sun into the motor's small bearings. V2 adds the output flange, the retainer and the ring's shoulder so every load direction ends in the ring and base.

- **Ring sits directly on the base:** the first print left a 2 mm gap, so the ring was extended 2 mm at the bottom and the sun made 2 mm taller to match. The CAD was updated to the as-built stack.
- **Firmware details:**
    - Timing uses SimpleFOC's own timer function. On the Uno, the shield's pins 5 and 6 sped up the Arduino's millisecond timer; on the Mega that no longer happens, but the code works on both.
    - The current-sensor check at start-up runs at 0.3 V. At SimpleFOC's usual 1.0 V it drives about 8 A, past the sensors' range.
    - The analog-to-digital converter runs 8× faster than the Arduino default, so reading the two sensors takes about 30 µs instead of about 220 µs.
    - SimpleFOC's serial commander holds at most 20 commands, so all current-loop settings share the single Q command. All 20 are now used (S was the last), so a new command means merging existing ones the same way. The firmware reads each line itself and upper-cases the command letter before passing it on, which is how lower case works without extra commands.

14. Project status

- **Done (October 29):** gearbox printed and assembled; motor recalibrated and tuned; gearbox tested under closed-loop control; output-angle (B) and zero (Z) commands added; CAD updated to match the build.
- **Done (October 30):** moved to an Arduino Mega; current sensing enabled with measured-current FOC; motor plus driver resistance and back-EMF measured; feedforward added; re-tuned with the gearbox, with step tests passing; holding jitter cut from about 7° to 0.5° at the motor by turning the current correction loop off; telemetry off at start-up; commands accept lower case; H or ? prints the command list; S sets the output speed; QV sets the voltage cap, and C and QV have safe maximums (4 A, 3 V).
- **Done (November 1):** V2 bearing redesign modeled and checked in CAD: output flange, bearing retainer, carrier and ring updated.
- **Done (November 2):** V2 printed and assembled; the output feels solid when pushed, pulled and rocked. Tested with unchanged firmware: 60° and 90° output steps settle in 0.3–0.6 s with no overshoot and end within 0.1°; holding jitter about 1° peak to peak at the motor (0.2° at the output). A slow full turn needs about 0.35 A, with a tighter zone between about 90° and 180° of output angle in both directions. Calibration arm designed (PlanetaryGearBox 5010-Arm.3mf). Web console built (Code/WebApp): commands, live charts, a 3D view, contact detection and the torque calibration; tested with its built-in simulator, not yet on the real joint.
- **Done (November 7–8):** Arm A1 designed and printed (straight sides with round ends, the flange pattern at both ends), plus a scale pusher. Output torque calibrated against a kitchen scale in three increasing sweeps: 0.099 N·m per amp above 0.87 A of friction (section 12). Firmware power-up limits raised to 3.5 A and 3.0 V. Web console tried on the real joint and redesigned: Drive tabs, a Force readout, Log and Setup tabs, an explanation button on every card, and a 3D model of Arm A1 that follows the arm even when it is turned by hand.
- **Changes since the first print:**

    - Rotor adapter: 4.5 mm screw grip, 7.7 × 2.7 × 4 mm key.
    - Sun: φ4.8 pin, 12.5 mm of teeth (+2 mm version).
    - Ring: 2 mm taller at the bottom (30 mm overall).
    - Motor: replaced with a spare after the first one's bell rubbed its windings.
    - Controller: Arduino Uno WiFi R3 replaced by an Arduino Mega 2560.
    - V2: output bearing clamped on all four faces (new flange and retainer, carrier and ring reprinted). The output face is 5 mm higher.
    - Arm A1 (36 mm wide, 6 mm thick) and a scale pusher replace the first calibration arm; firmware power-up limits raised to 3.5 A and 3.0 V.
- **Next:**
    - Optional: even out the tight zone by loosening and re-tightening the retainer and flange screws evenly.
    - Run Learn free motion on the real joint, then check that normal moves do not show contact.
    - Map torque across output angles, including the tight zone between about 90° and 180°, with the calibration rig.
    - Measure backlash.
    - A homing routine, so the output zero survives power-off.
    - Design the joint housing and output link.
    - A higher-current driver for more torque.